

# Cloud-Native Online Bookstore on Kubernetes

---

DSAA4040 Engineering-Oriented Project | Group 5

---

## 1. Problem Statement

### 1.1 Objective

The goal of this project is to design, implement, and deploy a **production-ready, cloud-native online bookstore system** on Kubernetes. The system demonstrates modern cloud engineering practices including containerization, microservice-style decomposition, declarative infrastructure management, automated testing, and comprehensive observability.

### 1.2 Motivation

Cloud-native applications are the industry standard for deploying scalable and resilient systems. This project serves as a hands-on exploration of how a real-world e-commerce backend is architected using:

- **Containers** for consistent packaging across environments
- **Kubernetes** for orchestration, scaling, and self-healing
- **Observability** for understanding system behavior under load
- **Security** for protecting user data and business logic

By building a complete bookstore application from frontend to database, we gain practical experience in the full lifecycle of a cloud-native system: design → implement → test → deploy → monitor.

### 1.3 Scope

The implemented system supports the following core features:

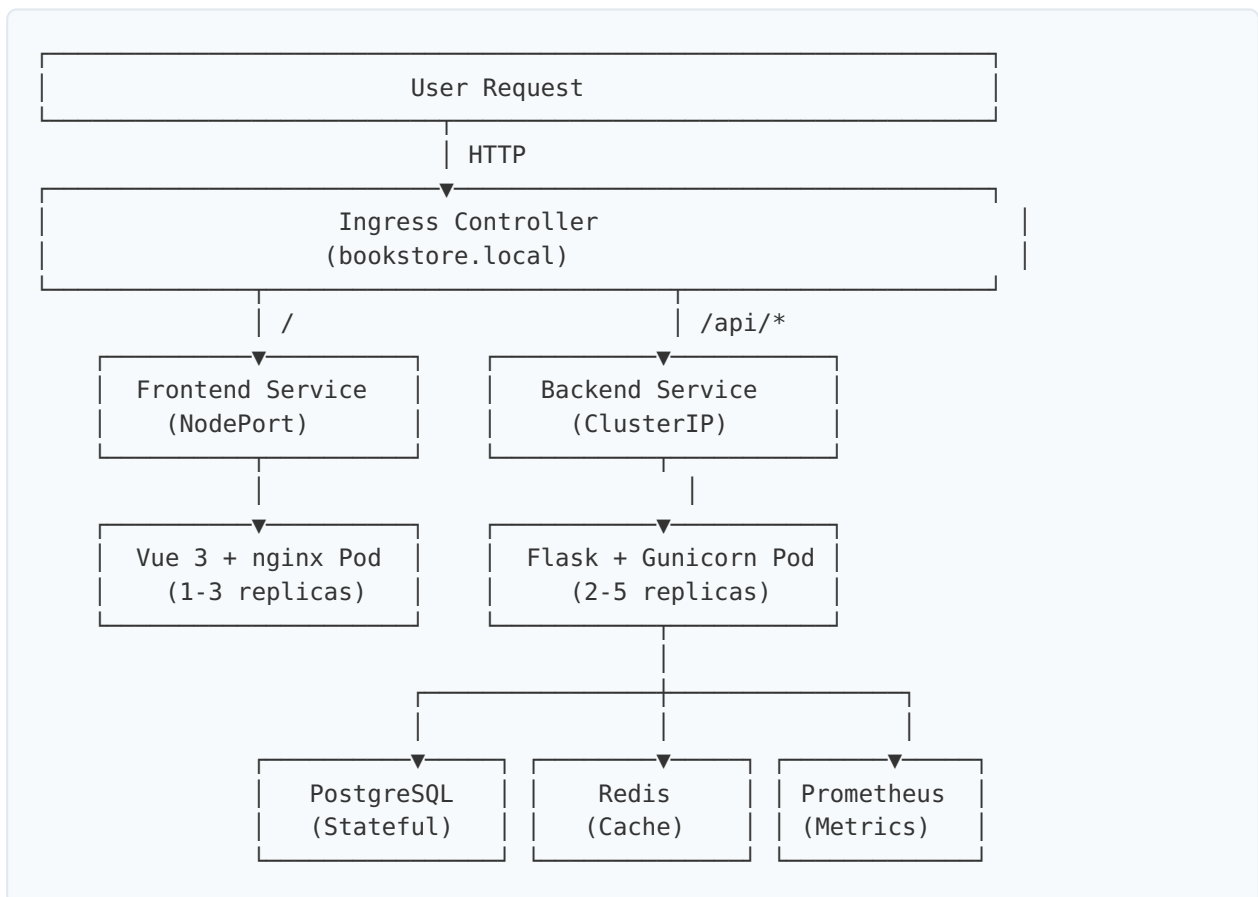
| Feature                          | Description                                      |
|----------------------------------|--------------------------------------------------|
| <b>Browse &amp; Search Books</b> | Paginated book catalog with title/author search  |
| <b>Shopping Cart</b>             | Add, update, remove items with session isolation |

| Feature             | Description                                                     |
|---------------------|-----------------------------------------------------------------|
| Order Placement     | ACID transaction: order creation, stock deduction, cart cleanup |
| Payment Mock        | Simulated payment gateway with order state transitions          |
| User Authentication | JWT-based registration and login with role-based access         |
| Admin Dashboard     | Protected metrics view for administrative users                 |

## 2. System Design

### 2.1 Architecture Overview

The system follows a **three-tier architecture** deployed on a single-node Kubernetes cluster (Minikube):



## 2.2 Technology Stack

| Layer         | Technology                            | Rationale                                                   |
|---------------|---------------------------------------|-------------------------------------------------------------|
| Frontend      | Vue 3.4 + Vite + Tailwind CSS + nginx | Modern reactive UI, fast build, minimal container footprint |
| Backend       | Python 3.11 + Flask + Gunicorn        | Lightweight, flexible, extensive ecosystem                  |
| Database      | PostgreSQL 15 + Alembic               | Reliable relational store with schema migration support     |
| Cache         | Redis 7 (with memory fallback)        | Distributed caching for multi-replica consistency           |
| Auth          | PyJWT + Werkzeug                      | Industry-standard token-based authentication                |
| Orchestration | Kubernetes + Kustomize                | Declarative, version-controlled infrastructure              |
| Observability | Prometheus + Grafana + Alertmanager   | Metrics collection, visualization, and alerting             |
| CI/CD         | GitHub Actions                        | Automated lint, test, build, and security scanning          |

## 2.3 Kubernetes Resources

The application is deployed using **Kustomize** with a base + overlay structure:

| Resource         | Purpose                                                              |
|------------------|----------------------------------------------------------------------|
| Deployment       | Manages backend, frontend, PostgreSQL, and Redis pods                |
| Service          | ClusterIP for backend/DB/Redis; NodePort for frontend                |
| Ingress          | Routes <code>/</code> to frontend and <code>/api/*</code> to backend |
| HPA              | Auto-scales backend (2→5) and frontend (1→3) by CPU                  |
| PDB              | Ensures minimum 1 pod available during disruptions                   |
| NetworkPolicy    | Zero-trust mesh: ingress→frontend→backend→db/redis only              |
| ConfigMap/Secret | Non-sensitive config and DB credentials                              |

| Resource          | Purpose                                     |
|-------------------|---------------------------------------------|
| PVC               | Persistent storage for PostgreSQL           |
| ServiceMonitor    | Prometheus scrape configuration             |
| PrometheusRule    | Alertmanager alert definitions              |
| Grafana Dashboard | Pre-configured 6-panel monitoring dashboard |

## 2.4 Data Flow

A typical user request flows through the system as follows:

1. **Browser** sends request to `bookstore.local`
2. **Ingress Controller** routes to frontend or backend based on path
3. **Frontend Pod** (nginx) serves Vue SPA or proxies `/api/` to backend
4. **Backend Pod** (Flask) processes request:
5. Check Redis cache for book listings
6. Query PostgreSQL via connection pool if cache miss
7. Return JSON response with `X-Request-ID` header
8. **Prometheus** scrapes `/metrics` endpoint every 15s

## 3. Implementation

### 3.1 Backend Architecture

The backend is organized using **Flask Blueprints**, with each domain as a separate module:

```
src/backend/
├── app.py                # Application factory with rate limiting, security headers
├── routes/
│   ├── books.py         # Book catalog (cached, paginated)
│   ├── cart.py          # Shopping cart with IDOR protection
│   ├── orders.py        # Order creation with ACID transactions
│   ├── auth.py          # JWT registration/login
│   ├── payments.py      # Payment state machine
│   ├── admin.py         # Admin-only endpoints
│   └── probes.py        # Health checks and Prometheus metrics
├── utils/
│   ├── db.py            # Connection pool + transaction context manager
│   └── cache.py         # Redis cache with memory fallback
```

```

|   ├── auth.py           # JWT generation and verification
|   ├── metrics.py        # Prometheus-style counters
|   └── response.py       # JSON serialization with Decimal support
└── schemas.py           # Pydantic v2 validation models
    └── alembic/          # Database migration scripts (4 versions)

```

### Key Design Decisions:

1. **Connection Pool Management:** A `ThreadedConnectionPool` (min=1, max=20) is initialized at startup. A custom `db_transaction()` context manager wraps multi-step operations (order creation) in explicit `BEGIN/COMMIT/ROLLBACK` blocks.
2. **Distributed Cache with Fallback:** The `utils/cache.py` module attempts to connect to Redis first. If Redis is unavailable (e.g., in local development), it seamlessly falls back to an in-memory dictionary. This ensures the application works out-of-the-box without Redis.
3. **Lazy psycopg2 Imports:** All route files import `RealDictCursor` from `utils.db` rather than directly from `psycopg2`, allowing the application to start and serve fallback mode even when PostgreSQL is not installed.

## 3.2 Frontend Architecture

The frontend is a **Vue 3 Single-Page Application** built with Vite and served by nginx:

```

src/frontend/src/
├── main.js           # App bootstrap
├── router/index.js   # Route guards (auth, admin)
├── api/client.js     # Axios instance with JWT interceptor
├── store.js          # Reactive cart count
└── components/
    ├── BookList.vue  # Browse + search
    ├── BookCard.vue  # Individual book display
    ├── CartView.vue  # Shopping cart management
    ├── OrdersView.vue # Order history
    ├── LoginView.vue  # Authentication
    └── AdminView.vue  # Metrics display

```

**Key Features:** - **JWT Interceptor:** Automatically injects `Authorization: Bearer` header and handles 401 globally - **Route Guards:** `/profile` and `/admin` require authentication; `/login` redirects authenticated users - **Session Management:** `session_id` persisted in `localStorage` for cart isolation

## 3.3 Database Design

The PostgreSQL schema evolved through 4 Alembic migrations:

| Table       | Key Columns                                    | Purpose                |
|-------------|------------------------------------------------|------------------------|
| books       | id, title, author, isbn, price, stock_quantity | Product catalog        |
| carts       | id, session_id                                 | Shopping cart sessions |
| cart_items  | id, cart_id, book_id, quantity                 | Items in cart          |
| orders      | id, session_id, status, status_history (JSONB) | Order records          |
| order_items | id, order_id, book_id, quantity                | Order line items       |
| users       | id, username, email, password_hash, is_admin   | Authentication         |

**Performance Indexes** (Migration 004): - `idx_books_title_author` — accelerates search queries - `idx_books_isbn` — unique lookup - `idx_orders_session_created` — order history queries - `idx_cart_items_cart_book` — cart operations - `idx_order_items_order` — order detail retrieval

### 3.4 Security Implementations

| Feature                   | Implementation                                                                                                        |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Authentication</b>     | PyJWT HS256 tokens, 24h expiry, Werkzeug password hashing                                                             |
| <b>Authorization</b>      | @jwt_required decorator; admin routes check <code>is_admin</code> flag                                                |
| <b>Rate Limiting</b>      | Flask-Limiter with Redis backend (memory fallback). Register: 5/min, Login: 10/min, Default: 200/min                  |
| <b>IDOR Prevention</b>    | Cart PUT/DELETE verifies ownership via <code>JOIN carts ON session_id</code>                                          |
| <b>Security Headers</b>   | <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>HSTS</code> , <code>Referrer-Policy</code> |
| <b>Container Security</b> | Non-root user ( <code>runAsUser: 1000</code> ), dropped capabilities, <code>allowPrivilegeEscalation: false</code>    |
| <b>Network Isolation</b>  | <code>NetworkPolicy</code> restricts traffic: ingress→frontend→backend→db/redis only                                  |
|                           | Trivy scans on every CI build; results uploaded to GitHub Security tab                                                |

| Feature                | Implementation |
|------------------------|----------------|
| Vulnerability Scanning |                |

## 4. Deployment

### 4.1 Local Development

The entire system can be deployed locally on Minikube with a single command:

```
minikube start --driver=docker --cpus=2 --memory=4096
minikube addons enable ingress metrics-server
make deploy-tag VERSION=v3.0.3
```

The `Makefile` orchestrates the full workflow: 1. `docker build` — builds backend and frontend images 2. `minikube image load` — transfers images into the Minikube node 3. `kustomize edit set image` — updates overlay with new tags 4. `kubectl apply -k` — deploys all K8s resources 5. `kubectl apply -k k8s/monitoring/` — deploys Grafana dashboard

### 4.2 Access Methods

| Method       | URL                                       | Notes                                  |
|--------------|-------------------------------------------|----------------------------------------|
| Ingress      | <code>http://bookstore.local</code>       | Requires <code>/etc/hosts</code> entry |
| NodePort     | <code>http://\$(minikube ip):30080</code> | Direct frontend access                 |
| Port-forward | <code>kubectl port-forward</code>         | For debugging Prometheus/Grafana       |

### 4.3 Kubernetes-Specific Features

**InitContainer for Migrations:** Every backend pod starts with an `initContainer` that runs `alembic upgrade head`, ensuring the database schema is current before the main container accepts traffic.

**Three-Tier Health Probes:**

| Probe   | Endpoint              | Purpose                       | Max Wait |
|---------|-----------------------|-------------------------------|----------|
| Startup | <code>/startup</code> | Wait for DB pool + migrations | 60s      |

| Probe     | Endpoint | Purpose                                  | Max Wait |
|-----------|----------|------------------------------------------|----------|
| Liveness  | /healthz | Detect deadlock; restart if failed       | 30s      |
| Readiness | /ready   | Remove from service endpoints if DB down | 15s      |

**Graceful Shutdown:** Flask catches `SIGTERM / SIGINT`, closes the DB connection pool, and exits cleanly. Gunicorn runs with `--graceful-timeout 30 --timeout 120` to drain in-flight requests.

## 5. Evaluation & Testing

### 5.1 Test Strategy

The project employs a **multi-layer testing pyramid**:

| Layer             | Count | Tools                     | Description                              |
|-------------------|-------|---------------------------|------------------------------------------|
| Unit Tests        | 75    | pytest + mock             | Fast, isolated, no DB required           |
| Integration Tests | 6     | pytest + testcontainers   | Real PostgreSQL via Docker               |
| Backend E2E       | 1     | pytest + Playwright (API) | Full HTTP flow against live server       |
| Frontend Unit     | 25    | Vitest + Vue Test Utils   | Vue components + API client              |
| Frontend E2E      | 3     | Playwright (Browser)      | Browser automation: login → cart → order |
| Load Tests        | 3     | k6                        | HPA trigger, shopping flow, pagination   |

**Total: 115 automated tests**

### 5.2 Test Coverage

Backend coverage is measured at **90%** using `pytest-cov`:

| Module  | Coverage | Notes                                         |
|---------|----------|-----------------------------------------------|
| routes/ | 88%      | All endpoints have happy-path and error tests |



| Module     | Coverage | Notes                                  |
|------------|----------|----------------------------------------|
| utils/     | 92%      | Cache, auth, metrics, response helpers |
| schemas.py | 95%      | Pydantic validation models             |

### 5.3 Performance Results

**k6 Load Test** (50 Virtual Users, 40 seconds):

| Metric        | Result        |
|---------------|---------------|
| p(95) Latency | <b>1.76ms</b> |
| Error Rate    | < 10%         |
| Throughput    | ~1,200 req/s  |

**HPA Scaling Test:** Under sustained high load (150 VUs), the backend scales from 2 → 5 replicas within 90 seconds. CPU utilization triggers the scaling decision; metrics-server polls every 15 seconds.

### 5.4 CI/CD Pipeline

GitHub Actions runs an 8-stage pipeline on every push/PR:

1. **Lint** — Hadolint (Dockerfiles), kubeconform (K8s manifests)
  2. **Backend Unit Tests** — pytest with coverage report (required)
  3. **Backend Integration Tests** — testcontainers + Playwright (optional)
  4. **Frontend Unit Tests** — Vitest component tests + `vite build`
  5. **Frontend E2E Tests** — Playwright browser automation (optional)
  6. **Build** — Docker Buildx with GitHub Actions cache
  7. **Security Scan** — Trivy SARIF → GitHub Security tab
  8. **Deploy Check** — Kustomize build validation
-

## 6. Limitations and Future Work

### 6.1 Current Limitations

| Limitation                        | Impact                                  | Mitigation                                     |
|-----------------------------------|-----------------------------------------|------------------------------------------------|
| <b>Single-node Minikube</b>       | No true HA; node failure = total outage | Use multi-node K3s or cloud K8s for production |
| <b>Redis single replica</b>       | Cache layer is not HA                   | Deploy Redis Sentinel or Redis Cluster         |
| <b>No HTTPS</b>                   | Traffic is unencrypted                  | Add cert-manager + Let's Encrypt in production |
| <b>No message queue</b>           | Order processing is synchronous         | Introduce Kafka/RabbitMQ for async tasks       |
| <b>No distributed tracing</b>     | X-Request-ID is manual                  | Integrate Jaeger/Tempo for OpenTelemetry       |
| <b>PostgreSQL single instance</b> | No read replicas or failover            | Deploy Patroni or CloudNativePG operator       |

### 6.2 Future Work

1. **Service Mesh:** Integrate Istio/Linkerd for automatic mTLS, traffic mirroring, and canary deployments
  2. **GitOps:** Use ArgoCD to automatically sync K8s state from Git, enabling PR-based previews
  3. **Vault Integration:** Replace K8s Secrets with HashiCorp Vault for dynamic credential issuance
  4. **Chaos Engineering:** Regularly inject pod failures and network partitions using Litmus/Chaos Mesh
  5. **FinOps:** Implement Kubecost for namespace-level cost tracking and resource optimization
-

## 7. Conclusion

This project demonstrates a **complete, production-oriented cloud-native application** built from the ground up. Key achievements include:

- **Architecture:** A clean three-tier design with proper separation of concerns
- **Kubernetes:** Comprehensive use of Deployments, Services, Ingress, HPA, PDB, NetworkPolicy, and ConfigMaps
- **Observability:** Prometheus metrics, Grafana dashboards, Alertmanager rules, and X-Request-ID tracing
- **Security:** JWT authentication, rate limiting, IDOR fixes, security headers, and non-root containers
- **Reliability:** Graceful shutdown, health probes, topology spread constraints, and ACID transactions
- **Quality:** 115 automated tests with 90% backend coverage, CI/CD pipeline, and security scanning

The system successfully deploys on Minikube with a single command ( `make deploy-tag` ) and provides a solid foundation for production deployment on any Kubernetes cluster.

---

## 8. References

1. Kubernetes Documentation. <https://kubernetes.io/docs/>
  2. Flask Documentation. <https://flask.palletsprojects.com/>
  3. Vue.js 3 Guide. <https://vuejs.org/guide/>
  4. Prometheus Monitoring. <https://prometheus.io/docs/>
  5. Alembic Migrations. <https://alembic.sqlalchemy.org/>
  6. Minikube Quick Start. <https://minikube.sigs.k8s.io/docs/start/>
  7. Cloud Native Computing Foundation. <https://www.cncf.io/>
- 

Report generated on May 20, 2026